Lab 3 - Inheritance, Polymorphism, and Class Hierarchies

This lab introduces inheritance and advanced object-oriented programming concepts through three progressive tasks. Students will learn class inheritance by extending the Person class to create a Student class, practice polymorphism and method overriding, and work with scalable data structures for realistic applications. The exercises progress from student management systems to vehicle hierarchies and animal classification, emphasizing inheritance relationships, code reusability, and polymorphic behavior.

Contents

Memorize

Submission Deadline

Deadline to upload the solutions for all tasks is Saturday, 11:59 pm before the lab date.

General Information

The following tasks are to be worked on in fixed teams of two. Each team member must be able to explain all solutions. Please submit only one solution for each team of two. The submission must be a PDF file in our Moodle room with the name and matriculation number. Solutions must be in digital format with intermediate steps and detailed explanations (no handwritten scans). You can use any tool or drawing program of your choice to create the diagrams. If you have questions or need support, use the forum in our Moodle room and help each other.

1. Task 1: Student Management with Inheritance

Extend your Java program from Lab 2 to demonstrate inheritance by creating a Student class that inherits from the Person class. This task introduces class inheritance, scalability, and advanced object management.

Extend your name management program as follows:

- Create a new `Student` class that inherits all aspects of the `Person` class
- Add a matriculation number attribute that starts at 1001 and increments automatically
- Use a constant `MAX_ANZAHL` to define maximum students (default: 500, scalable to millions)
- Implement proper encapsulation for the matriculation number (private with getter/setter)
- Extend the menu system:
  - Input 0: Exit program
  - Input `MAX_ANZAHL+1`: Display all student attributes
  - Input `MAX_ANZAHL+2`: Display maximum manageable students

Detailed requirements:

- Create a new class Students that inherits all aspects of the Person class in addition to the matriculation number. The matriculation number starts with 1001 and increases by one with each additional person. This matriculation number can also only be accessed via appropriate methods. Otherwise, everything remains as before, i.e., the program is terminated with input 0, with input `MAX_ANZAHL+1` all attributes of all students are displayed, and with `MAX_ANZAHL+2` the maximum number of students manageable by the program is output.

1.1. Preparation

First clarify the task by drawing the Student and Person classes according to UML notation, considering their inheritance relationship. Then define all necessary classes, methods, and variables in Java.

1.2. Assistance

The following code snippets demonstrate key inheritance concepts:

Basic inheritance structure:

```java
public class Person {
    protected String firstName;
    protected String lastName;
    protected int day, month, year;
    private static int personCount = 0;

    public Person(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
        personCount++;
    }

    // Getter and setter methods...
}

public class Student extends Person {
    private int matriculationNumber;
    private static int nextMatriculationNumber = 1001;

```

```
20      public Student(String firstName, String lastName) {
21          super(firstName, lastName); // Call parent constructor
22          this.matriculationNumber = nextMatriculationNumber++;
23      }
24
25      public int getMatriculationNumber() {
26          return matriculationNumber;
27      }
28  }
```

Using constants for scalability:

```java
1   public class StudentManager {
2       private static final int MAX_ANZAHL = 500;
3       private Student[] students = new Student[MAX_ANZAHL];
4
5       // Menu options based on MAX_ANZAHL
6       if (choice == MAX_ANZAHL + 1) {
7           displayAllStudents();
8       } else if (choice == MAX_ANZAHL + 2) {
9           System.out.println("Maximum students: " + MAX_ANZAHL);
10      }
11  }
```

2. Task 2: Vehicle Hierarchy with Polymorphism

Create a vehicle management system that demonstrates inheritance and polymorphism. This task will help you understand method overriding and polymorphic behavior.

Create a class hierarchy for different types of vehicles:

- Base class `Vehicle` with common properties (brand, model, year)
- Subclasses `Car`, `Motorcycle`, and `Truck` that extend Vehicle
- Each subclass should have specific attributes and override methods appropriately

Your program should:

1. Create a base `Vehicle` class with:
   - Protected attributes: brand, model, year
   - Constructor and getter methods
   - A `displayInfo()` method that shows basic vehicle information
   - An abstract or overridable `calculateMaintenanceCost()` method
2. Create subclasses that:
   - Add specific attributes (e.g., `numberOfDoors` for Car, `cargoCapacity` for Truck)
   - Override `displayInfo()` to include subclass-specific information
   - Implement `calculateMaintenanceCost()` with different formulas
3. Create an array of Vehicle objects containing different types
4. Use polymorphism to call methods on all vehicles in the array

2.1. Requirements

- Use inheritance with `extends` keyword
- Demonstrate method overriding with `@Override` annotation
- Create polymorphic behavior using Vehicle array containing different subclasses
- Use `protected` access modifiers appropriately

Note about `@Override`: The `@Override` annotation is a helpful tool that tells Java you intend to override a method from the parent class. While not strictly required, it's considered good practice because it helps catch errors at compile time. If you misspell a method name or get the parameters wrong, Java will give you an error instead of accidentally creating a new method.

2.2. Assistance

Basic vehicle hierarchy:

```java
public abstract class Vehicle {
    protected String brand;
    protected String model;
    protected int year;

    public Vehicle(String brand, String model, int year) {
        this.brand = brand;
        this.model = model;
        this.year = year;
    }

```

```
    public void displayInfo() {
        System.out.println(year + " " + brand + " " + model);
    }

    public abstract double calculateMaintenanceCost();
}

public class Car extends Vehicle {
    private int numberOfDoors;

    public Car(String brand, String model, int year, int doors) {
        super(brand, model, year);
        this.numberOfDoors = doors;
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Doors: " + numberOfDoors);
    }

    @Override
    public double calculateMaintenanceCost() {
        return (2025 - year) * 150.0; // Older cars cost more
    }
}
```

Polymorphic array usage:

```java
Vehicle[] vehicles = {
    new Car("Toyota", "Camry", 2020, 4),
    new Motorcycle("Honda", "CBR", 2021, 600),
    new Truck("Ford", "F-150", 2019, 1500)
};

for (Vehicle v : vehicles) {
    v.displayInfo(); // Calls overridden method
    System.out.println("Maintenance: $" + v.calculateMaintenanceCost());
}
```

3. Task 3: Animal Classification System

Create an animal classification system that demonstrates advanced inheritance concepts including abstract classes and interfaces. This task focuses on designing hierarchical relationships.

Design a system that models different types of animals with their behaviors:

- Abstract base class `Animal` with common properties
- Interface `Flyable` for animals that can fly
- Interface `Swimmable` for animals that can swim
- Concrete animal classes that implement appropriate interfaces

Your program should:

1. Create an abstract `Animal` class with:
   - Protected attributes: name, species, age
   - Abstract method `makeSound()`
   - Concrete method `displayInfo()`
2. Create interfaces:
   - `Flyable` with method `fly()`
   - `Swimmable` with method `swim()`
3. Create concrete animal classes:
   - `Bird` (extends Animal, implements Flyable)
   - `Fish` (extends Animal, implements Swimmable)
   - `Duck` (extends Animal, implements both Flyable and Swimmable)
   - `Dog` (extends Animal, no additional interfaces)
4. Demonstrate multiple inheritance through interfaces

3.1. Requirements

- Use abstract classes and methods
- Implement multiple interfaces in same class
- Override abstract methods in concrete classes
- Demonstrate interface polymorphism

3.2. Assistance

Abstract class and interface structure:

```java
public abstract class Animal {
    protected String name;
    protected String species;
    protected int age;

    public Animal(String name, String species, int age) {
        this.name = name;
        this.species = species;
        this.age = age;
    }

    public void displayInfo() {
```

```
13          System.out.println(name + " is a " + age + " year old " + species);
14      }
15
16      public abstract void makeSound();
17  }
18
19  public interface Flyable {
20      void fly();
21      default int getMaxAltitude() { return 1000; } // Default method
22  }
23
24  public interface Swimmable {
25      void swim();
26  }
27
28  public class Duck extends Animal implements Flyable, Swimmable {
29      public Duck(String name, int age) {
30          super(name, "Duck", age);
31      }
32
33      @Override
34      public void makeSound() {
35          System.out.println("Quack!");
36      }
37
38      @Override
39      public void fly() {
40          System.out.println(name + " is flying over the pond!");
41      }
42
43      @Override
44      public void swim() {
45          System.out.println(name + " is swimming in the water!");
46      }
47  }
```

4. Task 4: Collections in Java Programming

Enhance your student management system by implementing generic collections and data structures. This task introduces Java Collections Framework, generics, and advanced data manipulation techniques.

Create a comprehensive student management system using Java collections:

- Replace arrays with appropriate Collection class called ArrayList
- Add search and sorting functionality
- Create custom comparators for different sorting criteria

Your program should:

1. Convert your Student array to use `ArrayList<Student>`
2. Implement a `StudentRegistry` class that uses:
   - `ArrayList<Student>` for maintaining insertion order
4. Implement multiple sorting options with your favorite sorting algorithm (refer to the last lab):
   - Sort by name (alphabetical)
   - Sort by matriculation number
   - Sort by age (if birth date is available)
5. Add advanced search functionality with your favorite search algortihm:
   - Find students by partial name match
   - Find students by matriculation number range
   - Find students older/younger than specific age

4.1. Requirements

- Use appropriate Collection interfaces and implementations
- Implement generic methods with proper type parameters
- Use lambda expressions for predicates and comparators
- Demonstrate the difference between List, Set, and Map usage
- Handle duplicate prevention using Set characteristics

4.2. Assistance

Collection usage patterns:

```java
public class StudentRegistry {
    private List<Student> students = new ArrayList<>();

    public void addStudent(Student student) {
        students.add(student);
    }
}
```

5. Task 5: Design Patterns Implementation

Implement common design patterns to improve your student management system's architecture. This task introduces Singleton, Observer, and Factory patterns in a practical context.

Apply design patterns to create a robust and maintainable system:

- Singleton pattern for system-wide configuration
- Observer pattern for real-time notifications
- Factory pattern for creating different types of academic entities
- Strategy pattern for different grading systems

Your program should:

1. Implement `ConfigurationManager` using Singleton pattern:
   - Manage system-wide settings (max students, university info)
   - Ensure only one instance exists throughout the application
   - Provide thread-safe access to configuration data
2. Create Observer pattern for student events:
   - `StudentObserver` interface for notifications
   - `StudentSubject` for managing observers
   - Notify observers when students are added, removed, or modified
3. Use Factory pattern for academic entities:
   - `PersonFactory` that creates different types (Student, Professor, Staff)
   - `CourseFactory` for different course types (Lecture, Lab, Seminar)
4. Implement Strategy pattern for grading:
   - Different grading strategies (German grades, ECTS, Pass/Fail)
   - Allow runtime switching between grading systems

5.1. Requirements

- Implement at least three different design patterns
- Demonstrate loose coupling between components
- Use interfaces to define contracts between classes
- Show how patterns improve code maintainability and extensibility

5.2. Assistance

Singleton implementation:

```java
public class ConfigurationManager {
    private static volatile ConfigurationManager instance;
    private int maxStudents = 500;
    private String universityName = "HAW Hamburg";

    private ConfigurationManager() {}

    public static ConfigurationManager getInstance() {
        if (instance == null) {
            synchronized (ConfigurationManager.class) {
                if (instance == null) {
```

```
                    instance = new ConfigurationManager();
                }
            }
        }
        return instance;
    }

    public int getMaxStudents() { return maxStudents; }
    public void setMaxStudents(int max) { this.maxStudents = max; }
}
```

Observer pattern structure:

```java
public interface StudentObserver {
    void onStudentAdded(Student student);
    void onStudentRemoved(Student student);
    void onStudentModified(Student student);
}

public class StudentRegistry {
    private List<StudentObserver> observers = new ArrayList<>();
    private List<Student> students = new ArrayList<>();

    public void addObserver(StudentObserver observer) {
        observers.add(observer);
    }

    public void addStudent(Student student) {
        students.add(student);
        notifyObservers(observer -> observer.onStudentAdded(student));
    }

    private void notifyObservers(Consumer<StudentObserver> action) {
        observers.forEach(action);
    }
}
```

Factory pattern example:

```java
public abstract class PersonFactory {
    public static Person createPerson(String type, String firstName, String lastName) {
        switch (type.toLowerCase()) {
            case "student":
                return new Student(firstName, lastName);
```

```
6               case "professor":
7                   return new Professor(firstName, lastName);
8               case "staff":
9                   return new Staff(firstName, lastName);
10              default:
11                  throw new IllegalArgumentException("Unknown person type: " +
    type);
12          }
13      }
14  }
```

6. Task 6: Exception Handling and File I/O

Implement comprehensive error handling and data persistence for your student management system. This task covers exception handling, file operations, and data serialization.

Add robust error handling and file-based data persistence:

- Custom exception classes for domain-specific errors
- File I/O operations for saving and loading student data
- Data validation with appropriate exception handling
- Logging system for tracking system events

Your program should:

1. Create custom exception classes:
   - `StudentNotFoundException` for lookup failures
   - `InvalidMatriculationNumberException` for number conflicts
   - `StudentRegistryFullException` when capacity is exceeded
   - `DataPersistenceException` for file operation errors
2. Implement file-based persistence:
   - Save student data to CSV and JSON formats
   - Load student data from files with error recovery
   - Backup and restore functionality
3. Add comprehensive input validation:
   - Validate name formats, matriculation numbers, dates
   - Handle malformed input gracefully
   - Provide meaningful error messages to users
4. Implement logging system:
   - Log system events, errors, and user actions
   - Different log levels (INFO, WARNING, ERROR)
   - Configurable log output (console, file)

6.1. Requirements

- Create custom exception hierarchy with meaningful inheritance
- Use try-with-resources for proper resource management
- Implement both checked and unchecked exceptions appropriately
- Demonstrate exception propagation and handling at different levels
- Use proper file I/O with error recovery mechanisms

6.2. Assistance

Custom exception hierarchy:

```java
public class StudentManagementException extends Exception {
    public StudentManagementException(String message) {
        super(message);
    }

    public StudentManagementException(String message, Throwable cause) {
        super(message, cause);
    }
```

```
}

public class StudentNotFoundException extends StudentManagementException {
    public StudentNotFoundException(int matriculationNumber) {
        super("Student with matriculation number " + matriculationNumber + " not found");
    }
}

public class StudentRegistryFullException extends StudentManagementException {
    public StudentRegistryFullException(int maxCapacity) {
        super("Student registry is full. Maximum capacity: " + maxCapacity);
    }
}
```

File I/O with exception handling:

```java
public class StudentDataManager {
    public void saveToFile(List<Student> students, String filename)
            throws DataPersistenceException {
        try (PrintWriter writer = new PrintWriter(new FileWriter(filename))) {
            writer.println("MatriculationNumber,FirstName,LastName,Age");
            for (Student student : students) {
                writer.printf("%d,%s,%s,%d%n",
                    student.getMatriculationNumber(),
                    student.getFirstName(),
                    student.getLastName(),
                    student.getAge());
            }
        } catch (IOException e) {
            throw new DataPersistenceException("Failed to save student data", e);
        }
    }

    public List<Student> loadFromFile(String filename)
            throws DataPersistenceException {
        List<Student> students = new ArrayList<>();
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line = reader.readLine(); // Skip header
            while ((line = reader.readLine()) != null) {
                try {
                    Student student = parseStudentLine(line);
```

```
                    students.add(student);
                } catch (IllegalArgumentException e) {
                    System.err.println("Skipping invalid line: " + line);
                }
            }
        } catch (IOException e) {
            throw new DataPersistenceException("Failed to load student data",
e);
        }
        return students;
    }

    private Student parseStudentLine(String line) {
        String[] parts = line.split(",");
        if (parts.length != 4) {
            throw new IllegalArgumentException("Invalid data format");
        }
        // Parse and validate data, throw exceptions for invalid formats
        return new Student(parts[1], parts[2]);
    }
}
```

Input validation with exceptions:

```
public class InputValidator {                                    Java
    public static void validateName(String name) throws IllegalArgumentException
{
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Name cannot be empty");
        }
        if (!name.matches("[a-zA-ZäöüÄÖÜß\\s-]+")) {
            throw new IllegalArgumentException("Name contains invalid
characters");
        }
    }

    public static void validateMatriculationNumber(int number)
            throws InvalidMatriculationNumberException {
        if (number < 1001 || number > 999999) {
            throw new InvalidMatriculationNumberException(
                "Matriculation number must be between 1001 and 999999");
        }
    }
}
```

7. Lab Execution:

If your program is not yet working without issue, we will try to correct this during the course of the lab. With good preparation, this should not be a problem. Every student is required to be able to explain their thought process at the beginning of the lab. By the end of the lab, the task needs to be completed. Of course, we will support you, but your personal commitment must also be clearly recognizable! Julian Moldenhauer, Furkan Yildirim, and Emily Antosch wish you lots of fun and success!